

Arch: An AI-Native Hardware Description Language for Register-Transfer Clocked Hardware Design

SHUQING ZHAO

We present **Arch** (AI-native Register-transfer Clocked Hardware), a hardware description language designed from first principles for micro-architecture specification and AI-assisted code generation. Arch provides first-class constructs for pipelines, FSMs, FIFOs, arbiters, register files, CAMs, RAMs, synchronizers, counters, buses, multi-cycle threads, and linked-list data structures—structures that existing HDLs express only as user-defined patterns prone to subtle errors.

A central design choice is that clocks and resets are parameterized types (`Clock<D>`, `Reset<Sync|Async, High|Low>`) rather than ordinary nets, converting clock-domain crossing (CDC) and reset-domain crossing (RDC) analysis from external linter passes into compile-time typing rules. A reset’s domain is inferred from the registers that consume it, keeping the type signature simple while enabling RDC checking across five bug classes. Combined with tracking of bit widths, port directions, single-driver ownership, and combinational acyclicity, the type system catches multiple drivers, undriven ports, implicit latches, width mismatches, combinational loops, and unsynchronized crossings before any simulator runs. A `guard` clause on `reg` declarations captures the valid-gated-data pattern declaratively, catching the producer-side bug where a valid flag asserts before the data register has been written.

Every syntactic choice is governed by an *AI-generatability contract*: an LL(1) grammar, no preprocessor, a uniform declaration schema, named block endings, directional connect arrows, and a `todo!` escape hatch let LLMs produce structurally correct, type-safe Arch from natural-language specs without fine-tuning.

The compiler emits deterministic, lint-clean IEEE 1800-2017 SystemVerilog and auto-generates safety properties (FIFO no-overflow, counter range, FSM legal-state) verified with EBMC. An integrated simulator generates compiled C++ models and supports both C++ and Python testbenches via a cocotb-compatible API. Case studies cover an L1 data cache and an AXI DMA controller (Yosys/OpenSTA, Sky130). Across 443 in-scope problems from the VerilogEval and CVDP benchmarks, all Arch-generated designs are correct; the 2 non-passing results are attributable to reference-testbench bugs rather than design defects.

CCS Concepts: • **Hardware** → **Hardware description languages and compilation**; **Simulation and emulation**; • **Computing methodologies** → **Machine learning**.

Additional Key Words and Phrases: Hardware Description Language, HDL, AI-Generated Hardware, Compiler, CDC, RDC, Formal Verification, Simulation, SystemVerilog

ACM Reference Format:

Shuqing Zhao. 2026. Arch: An AI-Native Hardware Description Language for Register-Transfer Clocked Hardware Design. In *Proceedings of (Arch 2026)*. ACM, New York, NY, USA, 36 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Author’s Contact Information: Shuqing Zhao, arch.hdl.lang@gmail.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

1 Introduction

The hardware description language landscape is dominated by two IEEE standards—Verilog/SystemVerilog [1] and VHDL [2]—designed in the 1980s and 1990s, well before modern type theory, formal verification integration, or large language model (LLM)–assisted code generation became practical. While SystemVerilog has grown to encompass verification constructs (UVM), constrained randomization, and assertions (SVA), the core RTL subset retains characteristics that produce hardware bugs detectable only at simulation time: implicit latches from incomplete `if` coverage, multiply-driven nets from accidental concurrent assignments, unsynchronized clock-domain crossings from untracked clock tags, and width mismatches from implicit truncation and extension rules.

Modern alternatives—Chisel [3], SpinalHDL [4], Amaranth [5], and PyRTL [6]—embed RTL in host languages (Scala, Python) and provide varying degrees of parameterization and type safety. However, embedding inherits host-language complexity (JVM runtime, Scala implicits, Python dynamic typing), and micro-architectural constructs such as pipelines with hazard detection, arbiters with fairness policies, and clock-domain crossings remain user-defined library patterns, not compiler-verified first-class constructs.

Simultaneously, LLMs have demonstrated remarkable ability to generate code from natural-language descriptions [7, 8]. Yet hardware generation remains brittle: Verilog’s irregular syntax, context-dependent semantics, and invisible failure modes (e.g., inferred latches) make LLM-generated HDL unreliable without extensive post-generation verification. No existing HDL was designed with AI generatability as a first-class design constraint.

A key insight motivating Arch is that *the critical metric for AI-assisted hardware design is iteration latency*—the time from generating a candidate design to discovering whether it is correct. In traditional HDL workflows, many categories of bugs (latches, multiply-driven nets, width mismatches, CDC violations) are invisible until simulation, and simulation itself is slow: commercial event-driven simulators carry heavyweight runtimes because the language permits constructs (delta cycles, X/Z propagation, non-deterministic `always` ordering) that require dynamic resolution at every time step. An AI agent iterating on a hardware design pays this latency cost on every cycle of generate-compile-simulate. Shortening this iteration radius requires two complementary strategies: (1) moving error detection from simulation time to compile time, so that the agent receives immediate structured feedback without running a simulator, and (2) accelerating simulation itself, so that the bugs that *can* only be found at runtime are found faster.

Arch addresses these gaps simultaneously. Its contributions are:

- (1) **First-class micro-architecture constructs.** Pipeline, FSM, FIFO, arbiter, register file, synchronizer, CAM, RAM, counter, linked list, clock gate, and bus (with `handshake`, `tlm_method`, and `credit_channel` sub-constructs for protocol-level modeling) are language keywords with compiler-verified semantics, not library patterns. Hooks allow customizable behavior injection, and templates enforce structural contracts across modules (Section 4).
- (2) **Four-dimensional static type system with first-class clock and reset.** Bit widths, clock domains, port directions, and signal ownership are tracked simultaneously, and clocks and resets are themselves parameterized types (`Clock<D>`, `Reset<Sync|Async, High|Low>`) rather than ordinary 1-bit nets. A reset’s domain is inferred from the registers that consume it. This converts CDC and RDC analysis from

external linter passes into compile-time typing rules, with five classes of RDC violations now detected at compile time. Every mismatch is a compile-time error (Section 3).

- (3) **AI-generatability contract.** A uniform declaration schema, an LL(1) grammar, named block endings, directional connect arrows, and a `todo!` placeholder enable LLMs to produce correct Arch without fine-tuning (Section 5).
- (4) **Deterministic, lint-clean output.** The compiler emits one SystemVerilog file per top-level module with no latches, no X-propagation, no multiply-driven nets, and no implicit CDCs (Section 6).
- (5) **Integrated simulation toolchain.** The `arch sim` command independently generates compiled C++ models for cycle-accurate simulation with assertion evaluation, waveform output, and CDC randomization. Verilator [9] serves separately as the verification oracle for generated SystemVerilog correctness during compiler development. Language-level invariants (no combinational loops, single-driver rule, static clock schedules) further enable a future fully native LLVM IR path projected to achieve 15–60× speedup over event-driven simulators with structurally guaranteed deterministic parallel execution (Section 7).

2 Language Design

2.1 Design Philosophy

Arch is built on five principles, each enforced at the language level rather than by convention:

No Baggage. Arch has no host-language runtime. Every keyword maps directly to a hardware structure. Nothing is a library pattern behind operator overloading.

Strong Types. Bit widths, clock domains, port directions, and signal ownership are tracked statically. Every mismatch is a compile-time error.

Micro-Architecture First. Pipeline, FSM, FIFO, Arbiter, RegFile, RAM, CAM, Counter, Synchronizer, Bus (with `handshake`, `tlm_method`, and `credit_channel` sub-constructs), and Thread are first-class keywords with compiler-verified semantics.

AI-Generatable. Uniform schema, named block endings, English keywords, no braces, an LL(1) grammar (no backtracking, no multi-token lookahead), and a `todo!` escape hatch allow any LLM to produce valid Arch from a natural-language description without fine-tuning.

Predictable RTL. One Arch construct always produces the same SystemVerilog structure. Designers can audit every output line.

2.2 Block Structure

Every compound construct opens with a keyword-and-name header and closes with a matching `end keyword Name`. No braces are used anywhere in the language.

Listing 1. Universal block grammar

```
1 module Alu
2   // body
3 end module Alu
4
5 pipeline Decode
```

```

6  stage Fetch
7    // nested body
8  end stage Fetch
9  end pipeline Decode
10
11 fifo TxBuffer
12   // body
13 end fifo TxBuffer

```

This invariant eliminates brace-counting errors and enables named-ending matching: the compiler verifies that every `end` closure matches its opener by both keyword and name. Mismatches are immediate compile errors.

2.3 Universal Declaration Schema

Every first-class construct follows an identical four-section layout:

Listing 2. Universal schema

```

1 keyword Name
2  param NAME: const = value; // 1: params
3  param NAME: type = SomeType;
4  port name: in Type;        // 2: ports
5  port name: out Type;
6  // construct-specific body // 3: body
7  assert name: expr;         // 4: verify
8  cover name: expr;
9  end keyword Name

```

This regularity is fundamental to AI generatability: an LLM that learns the schema for one construct can apply it to all constructs without additional training.

3 Type System

The Arch type system enforces four independent safety dimensions simultaneously. A signal that satisfies all four is guaranteed correct-by-construction.

3.1 Primitive Types

Table 1 lists all primitive types. Each type carries hardware-relevant metadata tracked at compile time.

Table 1. Arch primitive types

Type	Width	Notes
Bit	1 bit	Raw logic bit
UInt<N>	N bits	Unsigned integer
SInt<N>	N bits	Two's complement signed
Bool	1 bit	Alias for UInt<1>
Clock<D>	1 bit	Domain-tagged clock
Reset<Sync Async, High Low>	1 bit	Sync or async; polarity defaults High; domain inferred from usage
Vec<T, N>	$N \times T $	Fixed-size array
struct	Σ fields	Named aggregate
enum	$\lceil \log_2 n \rceil$	Discriminated union

3.2 Bit-Width Safety

Every assignment, port connection, and arithmetic result is width-checked at compile time. There is no implicit truncation, zero-extension, or sign-extension anywhere in the language. Explicit conversion functions (`zext`, `sext`, `trunc`) are required:

Listing 3. Explicit width conversion

```
1 let a: UInt<8> = 255;
2 let b: UInt<16> = a.zext<16>(); // explicit
3 // let c: UInt<16> = a; // ERROR: width mismatch
```

Shift operations are likewise non-widening: `let wide: UInt<9> = a << 1;` is a compile error per IEEE 1800 §11.6.1, even though SystemVerilog accepts it silently. The designer must explicitly extend the operand (`a.zext<9>() << 1`) when bits are expected to grow, eliminating an entire class of subtle data-loss bugs.

3.3 First-Class Clock and Reset Types

A central design choice in Arch is that *clock and reset are first-class types in the type system, not bit signals*. Every clock is declared `Clock<D>` where `D` is a phantom domain parameter naming its frequency/source domain (e.g., `SysDomain`, `UsbDomain`). Every reset is declared `Reset<Sync|Async, High|Low>` carrying its synchronicity (`Sync/Async`) and polarity (`High/Low`, defaults `High`). A reset’s clock domain is inferred from the registers that consume it rather than carried as an explicit type parameter.

In SystemVerilog and VHDL, clocks and resets are ordinary 1-bit nets indistinguishable at the type level from any other wire. Whether two flip-flops belong to the same clock domain, whether a reset is synchronous or asynchronous, and whether a reset crossing has been properly synchronized are conventions enforced (at best) by external linters and CDC tools like Spyglass or Conformal CDC—tools that operate on netlists after synthesis, far removed from the source. Even modern HDLs that elevate type safety in other dimensions (Chisel, SpinalHDL) typically expose clock and reset only through implicit module-level handles, not as parameterized types that the type checker can reason about per-signal.

Treating clock and reset as parameterized types has three direct consequences that the rest of this section builds on:

- **Basic CDC checking is a typing rule, not a separate analysis pass.** A register declared in domain `A` that reads a signal driven from domain `B` is a type error—the same kind of error as assigning a `UInt<8>` to a `UInt<16>`—caught at compile time with no netlist post-processing and no possibility of the rule being silently disabled. The compiler also detects *reconvergent CDC paths* (where multiple individually synchronized bits of a source signal recombine in the destination domain and lose bit coherence) and warns when `kind ff` is used on multi-bit data. For designs with complex CDC topologies, Arch’s static checks should still be complemented by a commercial CDC tool run on the generated SystemVerilog. Section 3.4 describes the mechanism.
- **RDC checking falls out of the same framework.** Because a reset’s clock domain is inferred from the registers that consume it, the same type-checker pass that detects clock-domain crossings can detect reset-domain crossings. Five RDC bug classes are now detected at compile time: cross-clock-domain async reset (structural), cross-async-reset-domain data paths, reset-driven clock gating, reconvergent

synchronizers (RDC + CDC), and combiner-derived reset glitches at instantiation boundaries. Section 3.5 describes the mechanism.

- **Synchronizers are typed bridges.** A `synchronizer` declares its source and destination clock types explicitly: `port src_clk: in Clock<A>; port dst_clk: in Clock<B>`. Connecting it incorrectly is a type error. The synchronizer kind (`ff`, `gray`, `handshake`, `reset`, `pulse`) is part of the declaration, so the choice of synchronization mechanism is visible in the source rather than hidden in a tool configuration file.

This is the structural property that lets Arch convert what is conventionally an external verification problem (CDC analysis, RDC analysis) into a compile-time error message naming the offending register, signal, and domain pair. RDC checking is already implemented for five bug classes; CDC reconvergence detection is operational and continues to be strengthened.

3.4 Clock Domain Tracking

Every clock signal carries a `Clock<D>` domain tag. The compiler tracks domain membership for every register and combinational signal. Direct assignment across domains is a compile error:

Listing 4. Clock domain crossing via synchronizer

```

1 // ERROR: cross-domain direct assign
2 // comb usb_data = sys_data;
3
4 // Correct: explicit synchronizer
5 synchronizer SysToUsb
6   kind ff;
7   param STAGES: const = 2;
8   port src_clk: in Clock<SysDomain>;
9   port dst_clk: in Clock<UsbDomain>;
10  port data_in: in Bool;
11  port data_out: out Bool;
12 end synchronizer SysToUsb

```

The compiler detects cross-domain register reads and reports CDC violations, directing the designer to use a `synchronizer` (for individual signals) or an `async fifo` (for bulk data). Five synchronizer kinds are supported: `ff` (flip-flop chain), `gray` (gray-code for multi-bit counters), `handshake` (req/ack protocol), `reset` (async assert, sync deassert), and `pulse` (single-cycle pulse regeneration). CDC checking extends transitively across module instantiation boundaries.

3.5 Reset Domain Crossing (RDC)

A reset’s clock domain is *inferred from usage*—there is no explicit domain annotation on the `Reset` type; the type stays `Reset<Sync|Async, High|Low>`. The compiler treats each async reset signal as originating its own domain. Sync and reset-none flops are transparent to the analysis: they originate no domain themselves, just propagate whatever async domains reach their data input. Five classes of RDC violations are detected at compile time:

- (1) **Cross-clock-domain async reset (structural, phase 1).** An async reset signal is bound to one clock domain; reusing it in a second domain is flagged, even when the data paths in the two domains are independent. The fix is one synchronizer instance per receiving clock domain.

- (2) **Cross-async-reset-domain data path (data-flow, phase 2a).** Per-flop reach-set analysis catches async-reset flops driving sync-reset or reset-none flops whose reset is not the same signal—the classic metastability hazard where a downstream flop captures mid-deassert transients.
- (3) **Reset-driven clock gating (phase 2b).** Logic reaching a `clkgate` instance’s enable input from an async-reset flop causes the gate to glitch on reset events.
- (4) **Reconvergent synchronizers—RDC + CDC (phase 2c).** A single source signal routed through two or more synchronizer instances targeting the same destination domain produces outputs that settle on different cycles, leading to inconsistent state.
- (5) **Combiner-derived reset glitches at instantiation boundaries (phase 2d).** Combinational expressions on reset signals at `inst` connections (`rst <- (rst_a | rst_b)`) produce transient pulses on edge skew.

Per-module opt-out is available via `pragma cdc_safe;` (suppresses CDC checks and RDC phase 1) or `pragma rdc_safe;` (suppresses all RDC phases). The analysis currently operates on `module` constructs; cross-instance data flow tracing (phase 2d extension) and synchronizer/FIFO-internal path checking remain as future work (Section 13).

3.6 Single-Driver Rule

Every signal in Arch has exactly one driver. Multiple assignments to the same signal from different blocks are compile-time errors. This eliminates an entire class of simulation-time nondeterminism and enables lock-free parallel simulation (Section 7).

3.7 No Implicit Latches

The compiler verifies that every signal assigned in a `comb` block is assigned on *all* control paths. A missing `else` branch or incomplete `match` is a compile error, eliminating the most common source of unintended latches in synthesized hardware. Intentional latches require an explicit `latch on ENABLE` construct.

3.8 No Combinational Loops

The compiler builds a combinational dependency graph across all `comb` blocks, `let` bindings, and instance port connections, then performs a topological sort. Any cycle in this graph—a signal whose value transitively depends on itself within a single clock cycle—is rejected as a compile-time error with a path trace identifying the loop. This static guarantee eliminates a notorious source of simulation/synthesis divergence in SystemVerilog (where combinational loops may simulate but fail synthesis, or vice versa) and is the structural property that enables Arch’s bounded-settle native simulation (Section 7).

4 First-Class Micro-Architecture Constructs

Arch elevates recurring micro-architectural patterns from user-defined code to first-class language constructs with compiler-verified semantics.

4.1 Pipeline

A `pipeline` is a sequenced chain of `stage` blocks. The compiler generates inter-stage registers, stall propagation, and flush logic from declarative annotations; forwarding muxes are expressed as explicit `comb`

if/else blocks referencing cross-stage signals (Arch does not have a dedicated **forward** keyword—making forwarding explicit avoids hidden mux fanout and keeps the generated SystemVerilog auditable):

Listing 5. Pipeline with forwarding

```

1 pipeline IntPipe
2   param WIDTH: const = 32;
3   port clk: in Clock<SysDomain>;
4   port rst: in Reset<Sync>;
5   port data_in: in UInt<WIDTH>;
6   port result: out UInt<WIDTH>;
7
8   stage Fetch
9     reg instr: UInt<WIDTH> reset rst=>0;
10    seq on clk rising
11      instr <= data_in;
12    end seq
13  end stage Fetch
14
15  stage Execute
16    reg out_val: UInt<WIDTH> reset rst=>0;
17    // cross-stage reference to Fetch.instr
18    seq on clk rising
19      out_val <= Fetch.instr + 1;
20    end seq
21  end stage Execute
22
23  stall when Execute.out_val == 0;
24  flush Fetch when branch_mispred;
25  comb result = Execute.out_val;
26 end pipeline IntPipe

```

The compiler: (1) inserts pipeline registers between stages, (2) generates stall signals that back-propagate when any stage asserts **stall when**, (3) generates flush masks triggered by **flush** directives, and (4) tracks per-stage **valid_r** registers for pipeline occupancy. Cross-stage data references (e.g., **Fetch.instr** read from **Execute**) are resolved to the retimed pipeline register value. Forwarding muxes, when needed, are expressed as explicit **comb if/else** blocks.

4.2 Finite State Machine

An **fsm** is declared as a set of named **state** blocks with explicit transitions. A **default** block provides shared output assignments emitted before the state **case** statement, so states only override what differs:

Listing 6. FSM with default block and transitions

```

1 fsm Controller
2   port clk: in Clock<SysDomain>;
3   port rst: in Reset<Sync>;
4   port start: in Bool;
5   port busy: out Bool;
6   port done: out Bool;
7
8   default state Idle;
9
10  default
11    comb
12      busy = false;
13      done = false;
14    end comb

```



```

15   end default
16
17   state Idle
18     -> Active when start;
19   end state Idle
20
21   state Active
22     let busy = true;
23     -> Done when count_done;
24   end state Active
25
26   state Done
27     let done = true;
28     -> Idle;
29   end state Done
30 end fsm Controller

```

The `default state Idle` declaration sets the reset state. The `default` block assigns `busy = false` and `done = false` at the top of the generated `always_comb`; individual states override only the signals that differ (e.g., `Active` sets `busy = true`). The compiler generates one-hot or binary state encoding (selected by a `param` or compiler flag) and the transition `case` logic. If no transition fires in a given cycle, the FSM holds in the current state.

4.3 FIFO

A `fifo` is a first-class construct with compile-time-verified flow control. The designer specifies depth, width, and clock domain(s); the compiler generates counters, full/empty flags, and gray-code pointer CDC for dual-clock FIFOs:

Listing 7. Dual-clock FIFO

```

1  fifo AsyncBuf
2  param DEPTH: const = 16;
3  param TYPE: type = UInt<32>;
4  port clk_wr: in Clock<WriteDomain>;
5  port clk_rd: in Clock<ReadDomain>;
6  port push_valid: in Bool;
7  port push_ready: out Bool;
8  port push_data: in TYPE;
9  port pop_valid: out Bool;
10 port pop_ready: in Bool;
11 port pop_data: out TYPE;
12 port full: out Bool;
13 port empty: out Bool;
14 end fifo AsyncBuf

```

The `kind` keyword selects FIFO (default) or LIFO (stack) behavior. LIFO is restricted to single-clock operation.

4.4 Arbiter

An `arbiter` manages N requestors competing for M shared resources. The designer declares port counts and an arbitration policy (`round_robin`, `priority`, `lru`, `weighted<W>`, or a custom function via `hook`); the compiler generates grant logic, stall propagation, and fairness guarantees.

4.5 Bus and Handshake Channel

A **bus** is a reusable port bundle: a named, parameterized collection of signals with explicit directions from the initiator’s perspective. A module attaches a bus port as either **initiator** (directions as declared) or **target** (all directions flipped); the compiler flattens the bundle into individual ports at code-generation time (**axi.aw_valid** becomes **axi_aw_valid**). This eliminates the repetitive port blocks—typically hundreds of lines for an AXI interface—that a library-module approach inevitably duplicates at every instantiation site.

Within a **bus** body, the **handshake** sub-construct collapses a valid/ready/payload channel into a single declaration. It is a *compile-time sum type*: one keyword names the payload *role*, one variant name selects the flow-control shape, and the compiler derives every individual wire direction. Listing 8 shows an AXI-Lite bus written as three handshake channels:

Listing 8. AXI-Lite as three handshake channels

```

1 bus BusAxiLite
2   param ADDR_W: const = 32;
3   param DATA_W: const = 32;
4
5   handshake aw: send kind: valid_ready
6     addr: UInt<ADDR_W>;
7     prot: UInt<3>;
8   end handshake aw
9
10  handshake w: send kind: valid_ready
11    data: UInt<DATA_W>;
12    strb: UInt<DATA_W/8>;
13  end handshake w
14
15  handshake b: receive kind: valid_ready
16    resp: UInt<2>;
17  end handshake b
18 end bus BusAxiLite

```

The role keywords **send** and **receive**—deliberately chosen in preference to **in/out**—name the *payload* role rather than the direction of any single wire. A **send** channel produces the payload and drives **valid/req**, while receiving **ready/ack**; the wire directions follow mechanically. The target perspective then flips everything at the bus port, so a **send** channel on the initiator becomes a **receive** channel on the target with no extra syntax. The pervasive “I flipped valid and ready” bug class is eliminated by construction because the user never names individual wire directions.

Six variants cover the common flow-control shapes: **valid_ready** (AMBA AXI/ACE and general on-chip streaming with bidirectional backpressure), **valid_only** (strobos, interrupts, FIFO-fronted datapaths), **ready_only** (pull-model register-file reads), **valid_stall** (pipeline interlock where stall is the natural signal), **req_ack_4phase** (async/GALS with return-to-zero), and **req_ack_2phase** (async/GALS, low transition count). Canonical references: ARM IHI 0022 for **valid_ready**; Sparsø and Furber for the req/ack variants; Intel Avalon for **valid_only**.

Handshake declarations carry a two-tier safety story that reuses the verification infrastructure of Section 7.4. *Tier 2* auto-emits per-variant concurrent SVA: every **valid_ready** channel gets **_auto_hs_<port>_<ch>_valid_stable** (**((valid && !ready) | => valid)**), every **valid_stall** channel gets **_auto_hs_..._valid_stable_while_stall**, and every **req_ack_4phase** channel gets **_auto_hs_..._req_holds_until_ack**. Properties were verified on

Manuscript submitted to ACM

Verilator 5.034 `-assert` and EBMC 5.11. *Tier 1.5* targets the two payload-correctness bugs that protocol SVA does not catch: at simulation time, `-inputs-start-uninit` warns when a producer asserts `valid` without ever writing the payload (the uninit shadow bit is gated on `valid/req` so a testbench that hasn't yet driven `valid` stays silent); at compile time, `arch check` warns when a consumer reads `<port>.<payload>` outside an `if <port>.<valid>` guard. Stateful protocols—credit-based flow control, PCIe credit accounting—are handled by the `credit_channel` sub-construct (shipped), where the compiler owns the sender counter, the receiver FIFO, and the Tier-2 protocol SVA. For transaction-level modeling above the signal level, the `tlm_method` sub-construct (shipped, v1 blocking) provides method-call semantics on bus ports, with the compiler generating the wire protocol and state-machine lowering automatically.

4.6 Thread

A `thread` block is a sequential block that may span multiple clock cycles. It bridges the gap between `seq` (single edge, no implicit state) and `fsm` (explicit named states): the body reads as straight-line code with `wait` statements at points where the design must pause, and the compiler lowers each `wait` to a state boundary in a synthesizable FSM. This makes sequential protocols—bus handshakes, DMA descriptors, calibration sequences—directly expressible without hand-translating intent to states and transitions.

Listing 9. AXI write transaction as a thread

```

1  thread on clk rising, rst_n low
2    fork
3      aw_valid = 1;
4      aw_addr = addr_r;
5      wait until aw_ready;
6      aw_valid = 0;
7    and
8      w_valid = 1;
9      w_data = data_r;
10     wait until w_ready;
11     w_valid = 0;
12   join
13   b_ready = 1;
14   wait until b_valid;
15   b_ready = 0;
16   resp_r <= b_resp;
17 end thread

```

The construct supports a small operator menu sufficient for protocol-level code: `wait until cond` (state boundary that advances when `cond` is high), `do ... until cond` (same-cycle combinational drive—the body fires immediately when the enclosing state is entered, without the one-cycle pause of `wait until`), `wait N cycle` (counter-driven N-cycle pause), `fork ... and ... join` (parallel arms with done-bit registers per arm), `for i in 0..N` (loop counter with one body state per iteration, width inferred from the end expression's type), and `if/else` where either or both branches contain `wait` (lowered as a dispatch state with per-branch substates that rejoin afterward). `thread once` produces a one-shot thread that holds in a terminal state after completion; named threads (`thread WriteHandler ... end thread WriteHandler`) generate independent FSMs in the same module; `generate_for` unrolls a parameterized number of identical threads, each with its own state register. When multiple threads share a bus, a `resource axi_wr: mutex<round_robin>;` declaration plus `lock axi_wr ... end lock axi_wr` blocks generate a real `arbiter` construct (supporting

all policies: `round_robin`, `priority`, `lru`, `weighted`, and `custom-via-hook`), a grant-indexed mux on signals driven inside the lock, and stall logic on each thread's FSM; only the granted thread's values reach the shared signals. A static lock-order analysis warns when two threads lock the same pair of resources in opposite order.

Lowering. The compiler classifies every signal referenced inside any thread of a module as combinational-driven, sequential-driven, or read-only, then co-lowers the threads of that module into a single auto-generated submodule with the parent module retaining only its non-thread items. Each thread becomes an integer state register (`_t{i}_state`), a per-state combinational enable block, and a single merged `always_ff` block; counters for `wait N cycle` and `for`-loops are emitted alongside, with loop-counter width inferred from the end expression's type (e.g., `for i in 0..burst_len-1` where `burst_len: UInt<8>` generates an 8-bit counter). For each declared `resource`, the compiler synthesizes a real `arbiter` construct (supporting all policies: `round_robin`, `priority`, `lru`, `weighted`, and `custom-via-hook`), a grant-indexed mux on signals driven inside the lock, and per-thread stall logic. Single-resource lock is deadlock-free by construction. For multi-resource patterns, the compiler runs static lock-order analysis across all threads in the module and warns when two threads acquire the same pair of resources in opposite order.

Trailing seq assign merge. A `<=` assignment that immediately follows a `do..until` or `wait until` statement (with no intervening combinational logic or control flow) is automatically merged into the preceding state's exit condition by the compiler. This eliminates a dead cycle: the register write fires on the same clock edge as the handshake completion, rather than one cycle later.

Auto-emitted contract assertions. An off-by-default flag `-auto-thread-asserts` on `arch build`, `arch sim`, and `arch formal` causes the thread lowerer to additionally emit concurrent SVA properties anchored to the lowered `_t{i}_state` register: `_auto_thread_t{i}_wait_until_s{si}` for each `wait until (state==si && cond | => state==next)`, a stay/done pair `_auto_thread_.._wait_stay_s{si} / _.._wait_done_s{si}` for each `wait N cycle`, and per-branch `_auto_thread_.._branch_s{si}_b{bi}` for each `fork/join` arm. All are wrapped in `translate_off/on`, with reset polarity inverted to a not-in-reset guard. Each property is a corollary of the lowering equivalence proof for its construct: an assertion firing therefore evidences a compiler bug or a hand-edit of the lowered RTL rather than a user-program bug, complementing the verification story of Section 7.4 with a built-in self-check on the thread lowering itself.

4.7 Additional Constructs

Arch also provides first-class constructs for: `regfile` (multi-ported register files with read-during-write policies, flop and latch storage kinds), `ram` (single-port, simple-dual, true-dual, and ROM with configurable latency), `cam` (content-addressable memory with combinational binary match, dual write-port, and value payload), `synchronizer` (CDC crossing for individual signals with selectable policies: `ff`, `gray`, `handshake`, `reset`, `pulse`), `counter` (saturating, wrapping, gray-code, one-hot, and Johnson modes), `linklist` (singly/-doubly/circular linked-list data structures with per-operation FSM controllers), `clkgate` (integrated clock gating cell, latch-based or AND-based), and `pipe_reg` (N-stage flip-flop delay chain for retiming).

Each follows the identical four-section schema of Listing 2.

4.8 Hooks — Customizable Behavior Points

When a built-in construct policy does not fit, Arch provides **hook** declarations: named function bindings inside a construct that declare an expected signature and map it to a user-defined **function**. Hooks allow designers to inject custom combinational logic into compiler-generated control structures without abandoning the construct’s safety guarantees.

Listing 10. Custom arbiter policy via hook

```

1 function MyGrantFn(req_mask: UInt<4>,
2                   last_grant: UInt<4>,
3                   extra: UInt<8>) -> UInt<4>
4   let masked: UInt<4> = req_mask & (last_grant ^ 0xF);
5   let pick: UInt<4> = masked != 0 ? masked : req_mask;
6   return pick & (~pick + 1).trunc<4>();
7 end function MyGrantFn
8
9 arbiter CustomArb
10   policy MyGrantFn;
11   param N: const = 4;
12   port clk: in Clock<SysDomain>;
13   port rst: in Reset<Sync>;
14   port extra_port: in UInt<8>;
15   hook grant_select(req_mask: UInt<4>,
16                   last_grant: UInt<4>,
17                   extra_port: UInt<8>) -> UInt<4>
18     = MyGrantFn(req_mask, last_grant, extra_port);
19 end arbiter CustomArb

```

Hook arguments bind to internal signals generated by the construct (e.g., **req_mask**, **last_grant**) or to user-declared ports and params on the enclosing construct. The bound function is emitted inline inside the generated SystemVerilog module. A missing required hook is a compile error. Hooks are also used inside **template** contracts (Section 4.9) to specify required function bindings.

4.9 Templates — Interface Contracts

A **template** is a compile-time-only construct that defines a structural contract: a set of required params, ports, and hooks that any implementing module must provide. Templates emit no SystemVerilog—they exist purely to enforce structural conformance across modules, serving as the Arch equivalent of traits or interfaces in software languages.

Listing 11. Template declaration and implementation

```

1 template Arbiter
2   param NUM_REQ: const;
3   port clk: in Clock<SysDomain>;
4   port rst: in Reset<Sync>;
5   port grant_valid: out Bool;
6   hook grant_select(req_mask: UInt<4>) -> UInt<4>;
7 end template Arbiter
8
9 module MyArbiter implements Arbiter
10   param NUM_REQ: const = 4;
11   port clk: in Clock<SysDomain>;
12   port rst: in Reset<Sync>;
13   port req_mask: in UInt<4>;
14   port grant_valid: out Bool;

```

```

15 port grant_out: out UInt<4>;
16
17 hook grant_select(req_mask: UInt<4>) -> UInt<4>
18   = FixedGrant(req_mask);
19
20 let grant: UInt<4> = FixedGrant(req_mask);
21 comb
22   grant_valid = grant != 0;
23   grant_out = grant;
24 end comb
25 end module MyArbiter

```

A template body contains only declarations—params (without defaults), ports, and hook signatures. No logic, registers, or `comb/seq` blocks are permitted. A module opts into a template contract with the `implements` keyword, and the compiler validates conformance:

- **Param presence:** every template param must appear in the implementing module.
- **Port presence and direction:** every template port must appear with matching direction and type.
- **Hook binding:** every template hook must have a corresponding binding in the module.

The implementing module may declare additional params, ports, and logic beyond what the template requires—the template is a minimum contract, not an exhaustive specification. This enables library authors to define reusable contracts (e.g., “any arbiter must expose `grant_valid` and provide a `grant_select` hook”) while leaving full implementation freedom to the designer.

5 AI-Generatability Contract

Arch makes a hard design commitment: *a large language model that has read the specification and nothing else must be able to generate structurally correct, type-safe Arch from a plain-English hardware description.* Every syntactic choice serves this goal.

5.1 Design Decisions for AI

Uniform Schema. Every construct uses the identical param/port/body/verification layout (Listing 2). An LLM that learns the schema once applies it universally.

LL(1) Grammar. Arch’s grammar is strictly LL(1): at every point during parsing, the next single token unambiguously determines which production rule to apply. There is no backtracking, no multi-token lookahead, and no context-dependent parsing. Every construct is identified by its first keyword (`module`, `fsm`, `pipeline`, `port`, etc.); every closing is `end` followed by a single keyword token. By contrast, SystemVerilog requires unbounded lookahead and GLR or backtracking parsers to resolve ambiguities between type declarations, expressions, and module instantiations. The LL(1) property gives AI generators four advantages: (1) no syntactic traps—every token sequence either parses to exactly one AST or is caught immediately as a syntax error; (2) instant error localization—the parser never backtracks, so errors are reported at the exact offending token; (3) context-free understanding—any code snippet can be parsed in isolation without holding the entire file in context; and (4) predictable token budget—the uniform `keyword Name ... end keyword Name` pattern has no hidden costs from macro expansion or optional delimiters.

No Preprocessor. Arch deliberately omits a textual preprocessor. SystemVerilog inherits Verilog’s ‘`define`’, ‘`ifdef`’, ‘`include`’, and ‘`undef`’ macro layer, which is widely recognized as a major source of bugs in production

designs: macros are unscoped text substitution with no type checking, can be silently redefined across files, expand to text whose error messages point to the post-expansion line rather than the source, and create a parallel language layer that interacts unpredictably with the parser. Macros also defeat any context-free analysis (an LLM cannot know what `MY_WIDTH` expands to without the full preprocessor state) and are a notorious source of simulator/synthesizer disagreement. Arch covers each legitimate use case of `define` with a typed, scoped language construct:

- **Constants:** `param NAME: const = 32;` replaces `define NAME 32`. The constant is typed, scoped to its enclosing construct or package, and visible to the type checker.
- **Shared definitions across files:** `package` declarations group types, enums, constants, and functions into a named namespace; other files import them with `use PkgName::*`. This replaces shared header files included via `include`, and unlike header files, packages have well-defined import semantics with no order-dependence or double-inclusion problems.
- **Conditional structure:** `generate_if` performs compile-time selection of ports, instances, and logic based on `param` values, replacing `ifdef` blocks. The conditional is part of the AST, not a preprocessor directive, so the type checker sees both branches and the choice is visible to tooling. Crucially, `generate_if` can *add or remove ports*: SystemVerilog’s `generate` runs after the module’s port list is fixed, so SV designers are forced to use `ifdef` around port declarations whenever a configurable interface is needed—a textbook case where the macro layer is not a stylistic choice but the only available mechanism. Arch’s pre-elaboration `generate` eliminates this forced use of macros entirely (see Section 8).
- **Code reuse / templating:** `function` declarations (for combinational expressions), `template` (for interface contracts), and `hook` (for injecting custom logic into compiler-generated constructs) cover the patterns that multi-line macros are typically used for in SystemVerilog—without text substitution, without scope leakage, and with full type checking.
- **Configuration / build variants:** compile-time `param` overrides at instantiation provide build-time configuration without the textual fragility of `ifdef CONFIG_X`.

The result is that Arch source files have no separate preprocessing pass: what you see is what the parser sees, error messages point to the source line where the problem actually is, and an LLM (or human) can understand any snippet without simulating macro expansion in their head.

Named Block Endings. Every block closes with `end keyword Name`. The most common LLM failure mode in code generation—incorrect nesting—becomes a hard compiler error. The generator always knows exactly which block it is closing.

No Braces. The `keyword+name` header and `end keyword Name` are the sole delimiters. There is no redundant `{` to emit or forget.

Directional Connect Arrows. Port connections use `<-` (drive input from local) and `->` (read output into local). Direction is visible in the syntax itself; an LLM cannot silently reverse data flow.

todo! Escape Hatch. The `todo!` keyword compiles and type-checks but aborts at simulation runtime. This enables incremental AI-assisted design: generate a correct skeleton, then fill in logic section by section. Every intermediate state compiles, so the AI gets useful feedback on the parts it has confidently generated even while uncertain pieces remain unfilled.

Listing 12. Partial implementation with `todo!`

```

1 module Cache
2   port req: in CacheReq;
3   port resp: out CacheResp;
4   port mem_req: out MemReq;
5
6   // Confident: forward request to memory
7   comb
8     mem_req.addr = req.addr;
9     mem_req.valid = req.valid;
10  end comb
11
12  // Uncertain: eviction logic deferred
13  comb resp = todo!; end comb
14 end module Cache

```

This compiles cleanly. The compiler verifies widths, types, and port connections in the confident sections, and the AI (or human) can iteratively replace each `todo!` site with real logic, getting compile-time feedback at each step. This is the hardware analog of red-green-refactor in TDD: skeleton first, logic second.

5.2 Minimal AI Context

The effective AI context for Arch hardware design comprises three components:

Table 2. AI context window components

Component	Size	Purpose
Reference Card	~400 lines	Construct catalog, schema, type table
Design Intent	5–20 lines	Natural-language block description
Compiler Output	5–30 lines	Structured error feedback

The full specification is a reference document for human designers. The AI Reference Card is what an LLM assistant actually uses—it is optimized for construct lookup and schema recall, not for sequential human reading.

5.3 AI-Assisted Design Workflow

The practical workflow has four steps, typically completing in two to four iterations:

- (1) **Intent.** The designer describes the desired hardware block in natural language.
- (2) **Generation.** The AI identifies the Arch constructs, applies the universal schema, and emits source code, using `todo!` for uncertain logic.
- (3) **Compilation.** The designer runs `arch check`. The compiler emits zero errors, or precise typed errors.
- (4) **Correction.** Compiler errors are fed back to the AI, which corrects and re-emits. The compiler replaces the spec in the feedback loop.

This workflow exploits three properties: (1) the uniform schema means the AI need not choose between implementation strategies, (2) the compiler produces precise, structured errors sufficient for self-correction,

and (3) hardware intent maps unambiguously to Arch constructs (a FIFO is always `fifo`, a state machine is always `fsm`).

Minimizing Iteration Latency. The workflow is designed to minimize the time from code generation to correctness feedback. In a traditional Verilog flow, an AI agent must generate code, invoke a simulator, wait for simulation to complete (seconds to minutes for event-driven tools), and parse waveform or log output to identify failures—many of which are structural errors (latches, width mismatches, CDC violations) that could have been caught statically. In Arch, `arch check` runs in milliseconds and catches these errors at compile time with structured diagnostics that name the construct, signal, type mismatch, and suggested fix. The agent’s inner loop—generate, check, correct—completes without ever invoking a simulator for structural errors. When simulation *is* needed (for functional verification), `arch sim` generates compiled C++ models in the same performance class as Verilator, avoiding event-driven scheduling overhead and achieving the 10–50× speedup over interpreted simulators characteristic of compiled 2-state simulation. Together, compile-time error detection and compiled simulation compress the iteration radius that determines how quickly an AI agent—or a human designer—can converge on a correct design.

Test-Driven Development for Hardware Agents. A natural extension of this tight iteration loop is test-driven development (TDD) [10], an increasingly popular methodology for AI-agent-driven hardware design. Recent empirical work in software engineering has shown that providing LLMs with tests alongside problem statements measurably improves code-generation success rates on standard benchmarks [11]. In TDD, the agent is given a specification and a test suite *before* writing any implementation. The agent then iterates: generate a candidate design, run the tests, observe failures, and refine. The effectiveness of TDD is directly proportional to how fast each generate-test-feedback cycle completes.

With traditional event-driven HDLs, each TDD iteration requires invoking a heavyweight simulator—seconds to minutes per cycle—making agent-driven TDD impractical for all but the simplest designs. Arch enables practical hardware TDD through a two-tier feedback loop. The *inner loop* uses `arch check` (milliseconds) to eliminate structural errors before any simulation runs; the agent may complete several generate-check-correct cycles per second. The *outer loop* uses `arch sim` (compiled C++ models) to run the actual test suite for functional verification. Like Verilator-class compiled simulators, `arch sim` avoids the overhead of event-driven scheduling, achieving the 10–50× speedup over interpreted simulators characteristic of compiled 2-state simulation. This two-tier structure means the agent spends most of its iterations in the fast inner loop and invokes simulation only when the design is structurally sound—exactly the workflow that makes TDD viable at scale.

The VerilogEval and CVDP benchmarks (Section 11) were themselves completed using this pattern: each design was written from a natural-language spec, compiled with `arch check`, and validated against reference testbenches—a workflow that mirrors how an AI agent would operate in a TDD loop.

6 Compiler Pipeline

The Arch compiler transforms source through six phases:

Table 3. Compiler pipeline phases

Phase	Key Actions
Parse	Syntax, named-block matching
Elaborate	Parameter resolution, generic instantiation, type expansion
Resolve	Symbol table construction, cross-file name resolution
Type Check	Bit-width safety, clock-domain tracking, direction safety, single-driver rule
Lower	Pipeline hazard generation, FIFO implementation, arbiter logic, FSM encoding
Verify Emit	assert/cover $\rightarrow$ concurrent SVA with auto-generated safety properties; assume planned
SV Emit	One deterministic, lint-clean SV file per top-level module
Formal Emit	arch formal : direct AST $\rightarrow$ SMT-LIB2 bit-vector transition relation

6.1 Output Guarantee

The generated SystemVerilog is guaranteed free of:

- Unintentional latches—the compiler verifies all **comb** signals are assigned on every control path; intentional latches require an explicit **latch on ENABLE** construct.
- Multiply-driven nets—enforced by the single-driver rule.
- Unresolved high-Z outputs—every output has exactly one driver.
- X-propagation from uninitialized state—all registers declare an explicit reset policy (including opt-out via **reset none**); the **-check-uninit** simulation flag detects reads of uninitialized registers at runtime.
- Implicit clock-domain crossings—all CDCs are declared and synchronizer-wrapped.

Synthesis tools receive RTL that passes lint cleanly with no **translate_off** pragmas or vendor-specific attributes required. During compiler development, the generated SystemVerilog is systematically verified against Verilator [9]—the industry-standard open-source cycle-accurate simulator—to confirm behavioral equivalence between Arch source semantics and the emitted RTL.

6.2 Intermediate Representation

The current compiler is a multi-phase pipeline: `parse  $\rightarrow$  elaborate  $\rightarrow$  resolve  $\rightarrow$  type-check  $\rightarrow$  codegen`, transforming the AST directly to SystemVerilog text without a dedicated intermediate representation. The Resolve phase constructs the symbol table and handles cross-file name resolution. This architecture prioritizes simplicity and correctness for the initial release. A dedicated intermediate representation (AIR) and CIRCT/MLIR backend integration are planned as future work to enable multi-target output and optimization passes.

6.3 Toolchain Targets

The compiler currently supports three output targets: IEEE 1800-2017 SystemVerilog (no vendor primitives, compatible with any standard EDA tool), independently generated compiled C++ simulation models via `arch sim`, and direct SMT-LIB2 bit-vector transition relations via `arch formal` (Section 7.4) for bounded model checking with Z3, Boolector, or Bitwuzla. Planned targets include FPGA-specific SystemVerilog with BRAM/DSP primitive insertion and HTML documentation from `/// doc` comments.

7 Simulation and Verification

7.1 Integrated Simulation

The `arch sim` command provides integrated cycle-accurate simulation. The compiler independently generates a C++ model per construct (`VName.h`, `VName.cpp`), compiles it with `g++`, and executes the resulting binary—no external simulator is invoked. Supported constructs include modules, pipelines, FSMs, counters, RAMs, FIFOs, arbiters, synchronizers, register files, and linked lists. The simulation flow supports VCD waveform output (`-wave`), assertion evaluation, uninitialized-register detection (`-check-uninit`) with complementary `-check-uninit-ram` and `-inputs-start-uninit` for memory cells and undriven ports respectively, and CDC latency randomization (`-cdc-random`) for stress-testing synchronizer designs. Separately, Verilator [9] serves as the primary verification oracle during compiler development: every code generation change is validated by comparing Arch-emitted SystemVerilog behavior against Verilator’s independent interpretation of the same RTL.

7.2 Python Testbenches via a cocotb-Compatible API

In addition to C++ testbenches driven through the `-tb` flag, `arch sim` supports Python testbenches written against a cocotb-compatible API [15]. Invoking `arch sim -pybind -test tb.py Module.arch` produces a pybind11 wrapper over the generated C++ model and runs the Python testbench against it with no external simulator, iverilog, or VPI in the loop. A thin scheduler (`arch_cocotb`) drives the model tick-by-tick from an `asyncio` event loop, and a drop-in `cocotb` shim re-exports the supported API so that the same test file runs unchanged under either `arch sim -pybind` or a real cocotb flow—provided the test stays within the intersection of the two APIs (`@cocotb.test`, `RisingEdge`, `FallingEdge`, `Timer`, `ClockCycles`, `Clock`, `start_soon`, and `get_sim_time`).

Listing 13. Python testbench against `arch sim`

```

1 import cocotb
2 from cocotb.triggers import RisingEdge
3 from cocotb.clock import Clock
4
5 @cocotb.test()
6 async def test_reset(dut):
7     cocotb.start_soon(
8         Clock(dut.clk, 10, units='ns').start())
9     dut.rst_n.value = 0
10    dut.enable.value = 0
11    await RisingEdge(dut.clk)
12    dut.rst_n.value = 1
13    for _ in range(5):
14        await RisingEdge(dut.clk)

```

```
15  assert int(dut.count.value) == 0
```

Two characteristics distinguish this path from conventional cocotb setups running against an RTL simulator. First, timing is *deterministic*: writes from Python to `dut.port.value` set a field on the pybind11 C++ object directly and are visible to `model.eval()` on the very next tick, eliminating the VPI callback-ordering ambiguity that appears in iverilog or Verilator cocotb flows. Second, the values flowing across the boundary are *2-state*: signal values are integer-like and have no per-bit X/Z state, consistent with the sim model of ???. The consequences are explicit in the implementation—edges are detected by per-tick sampling rather than event callbacks, writes take effect immediately rather than in an NBA region, and there are no delta cycles—and are documented as known deltas from real cocotb. The integration trades VPI fidelity for deterministic timing and faster iteration, and benefits directly from the structural guarantees enumerated in ??? (no implicit latches, no combinational loops, single-driver, static DAG scheduling), which are what let a straight-line `eval()` loop substitute for an event-driven kernel without loss of cycle-accuracy.

7.3 Path to Native Compiled Simulation

Arch’s language-level invariants enable a future fully native simulation path that compiles designs directly to LLVM IR-based binaries, bypassing Verilator for higher throughput. This section describes the structural properties that make such compilation feasible and the expected performance characteristics.

7.3.1 Why Native Compilation is Feasible. Traditional Verilog simulators carry a heavyweight runtime because the language permits constructs requiring dynamic resolution. Arch eliminates every one at the language level, as shown in ???.

The compile-time DAG analysis statically determines a settle depth of 1 or 2 for each module: depth 1 when sub-instance inputs are driven directly, depth 2 when intermediate `comb` or `let` bindings in the parent feed instance inputs and a second pass is needed to propagate them. The generated code is therefore a fixed bounded loop, not a fixpoint iteration—unlike SystemVerilog’s delta cycles, which can iterate an unbounded number of times until convergence.

7.3.2 Expected Performance. Because Arch’s execution model compiles to a straight-line native loop with bounded settle and no dynamic dispatch, the projected simulation throughput is competitive with Verilator—currently the fastest open-source HDL simulator—and substantially exceeds event-driven commercial simulators. ??? summarizes the expected performance characteristics of the planned native backend.

SIMD vectorization is automatic for designs with wide `Vec<SInt<8>,N>` or `Vec<UInt<N>,M>` types—precisely the types used in systolic arrays and attention units in AI accelerator designs.

7.3.3 Limitations of 2-State Simulation. Arch’s 2-state model eliminates the structural sources of X by construction: there are no tri-state nets (no high-impedance Z), no multiple drivers (no contention X), no implicit latches (no unknown-enable X), and no `'x` literal in the source language. However, 2-state simulation has a well-known weakness: it can mask design bugs that 4-state simulation with X-propagation would catch—a phenomenon known as *X-optimism* [14]. ??? enumerates the X sources identified in the X-optimism literature and Arch’s current handling of each.

The first group—structural X sources—is the strongest argument for Arch’s design: these are eliminated by typing rules and compiler invariants, not detected after the fact. The second group—value-dependent X

Manuscript submitted to ACM

sources such as uninitialized RAM, out-of-bound indexing, and division by zero—was the honest weakness of 2-state simulation in earlier Arch releases and is the motivation for the detection mechanisms listed in the lower half of ???. Industry experience documented in [14] shows that out-of-bound array indexing in particular has caused post-silicon bugs (e.g., the audio-processing IP case study in that paper, where a bit-select with an unintended index value masked a state-machine bug for an entire chip generation).

Arch now detects each of these cases through a combination of three techniques. *Runtime abort* in the native C++ simulator: every `Vec<T,N>` index, bit-select, variable part-select (`val[start +:W] / val[start -:W]`), and division operator is wrapped in a bounds / non-zero check that aborts with a source location on violation, with no opt-in flag required. *Auto-emitted concurrent SVA* in generated SystemVerilog: the compiler inserts `_auto_bound_*` and `_auto_div0_* assert property` statements for every such operation in `seq` and `latch` contexts, wrapped in `synopsys translate_off/on` so they are consumed by simulation and formal tools but skipped by synthesis. *Compile-time rejection* where the value is statically known: `arch check` rejects constant-folded `A / 0` and `A % 0` in param defaults and `const let` initializers, catching the worst cases before any simulator runs. The auto-emitted properties have been verified with Verilator 5.034 `-assert` and EBMC 5.11, the latter reporting `PROVED` for structurally-safe cases such as `UInt<2>` indexing into `Vec<_,4>` or division by a non-zero expression (`den | 1`). For RAM cells, the `-check-uninit-ram` flag maintains a per-cell valid bitmap (cells initialized via an `init:` clause are pre-marked; ROMs are exempt). For undriven primary inputs, `-inputs-start-uninit` warns once per port on first read.

The guard clause. A specific refinement targets the most common case of intentionally reset-free state: the *valid-data pattern*, where a wide data register is deliberately left unreset (to save area and power) and a companion `Bool` valid flag gates all consumers. In plain SystemVerilog this pattern is hand-coded, and two failure modes are notoriously hard to catch: forgetting to qualify the read with the valid flag (consumer bug), and letting the valid flag assert before the data register has been written (producer bug). Arch captures both with a declarative annotation:

```
1 reg axi_rdata: UInt<512> guard axi_rvalid;
2 reg axi_rvalid: Bool reset rst => false;
```

The `guard VALID_SIG` clause documents that `axi_rdata` is intentionally reset-free and is only meaningful when `axi_rvalid` is high. It has four observable effects. First, it silences the false-positive `-check-uninit` warning at consumer read sites—consumers are expected to qualify the read with `if axi_rvalid`, so an unwritten-and-unused data register is not a bug. Second, it catches the exact producer bug the pattern exists to prevent: if the guard asserts (`axi_rvalid == true`) but the data register has never been written, `-check-uninit` emits a runtime warning. Third, it documents intent in the source, so a reader or reviewer can tell at a glance that the missing reset is deliberate rather than an oversight. Fourth, the same declarative intent supplies a don’t-care specification for *sequential equivalence checking* between a clock-gated design and its free-running reference. SEC flows comparing the two typically flag the gated data registers as miscompare points during gated cycles—the gated version holds a stale value while the free-running version updates (or vice versa)—even though the values are by construction unused when the guard is low. A `guard VALID_SIG` annotation captures precisely this “don’t-care when valid is low” contract at the source level, available to downstream tooling as an SVA `assume` constraint or vendor-specific waveform condition. The same annotation is available on `port reg` outputs. Combined with the `reset none / reset SIG=>VAL`

distinction, the **guard** clause turns a loose coding convention into a first-class language feature whose design intent is consumable by simulation, formal, and equivalence-checking tools alike.

For designs where pre-silicon X-optimism analysis is critical—particularly third-party IP integration and power-managed designs—we recommend complementing Arch simulation with the formal X-propagation methodology described in [14] applied to the generated SystemVerilog.

7.3.4 Parallel Simulation. Arch supports parallel cycle-accurate simulation exploiting three levels of parallelism:

- (1) **DAG-level:** independent nodes in the same topological level execute concurrently within a single clock cycle.
- (2) **Module-level:** independent module instances with no intra-cycle dependency execute on separate threads.
- (3) **Domain-level:** separate clock domains run on dedicated threads, synchronizing only at CDC crossings.

Determinism Guarantee. Parallel Arch simulation is guaranteed to produce bit-identical results to sequential simulation, regardless of thread count, OS scheduling, or memory ordering. This follows from three structural properties: (1) no write conflicts (single-driver rule), (2) register commit barriers (all next-state values computed before any update), and (3) DAG level barriers (level $N+1$ never starts before level N completes on all threads).

This is a structural theorem, not a best-effort property. A simulation passing on 1 core produces identical assertion results on 64 cores.

Table 7 shows expected parallel speedup across design types.

Table 7. Parallel simulation speedup

Design Type	8 cores	32 cores
Single-domain, deep chain	1.5–2×	2–3×
Wide parallel array	3–5×	5–8×
Multi-domain balanced	5–7×	10–18×
Multi-domain + wide arrays	6–8×	15–25×
AI accelerator (full)	7–9×	18–30×

7.4 Verification

Arch specifies three verification constructs that use the same types and signals as the design: **assert** for invariant checking, **cover** for reachability properties, and **assume** for environment constraints. **assert** and **cover** are fully implemented; **assume** is specified but not yet lexed. Both ship as concurrent SVA—**assert property** (@(posedge clk) *expr*) and **cover property** (...)—with an **implies** binary operator lowering to (!*a* || *b*). The compiler additionally auto-generates safety properties for every first-class construct: **_auto_no_overflow** and **_auto_no_underflow** on every FIFO, **_auto_count_range** on every counter, **_auto_legal_state** on every FSM, and per-state reachability plus per-transition covers so that unreachable states and untaken transitions surface without designer effort. All SVA is wrapped in **translate_off/on** for synthesis portability.

Property scope. An **assert** or **cover** body is a *combinational expression* evaluated at each clock edge under the construct’s **posedge clk** with **disable iff (rst)**. This covers the same-cycle safety fragment of SVA, including overlapping implication: **a implies b** is exactly SVA’s **a |-> b** and lowers to **(!a || b)** at each cycle. For multi-cycle properties, designers currently bind the temporal state into explicit shadow registers and assert on them—for example, a next-cycle acknowledge property is written as **reg req_d1: Bool reset rst => false;** inside a **seq** block updating **req_d1 <= req;** followed by **assert next_cycle_ack: req_d1 implies ack;**. Planned syntactic sugar for the post-v0.41 roadmap—**a |=> b** (non-overlapping implication), **##N a** (N-cycle delay), **past(expr, N)**, and **\$rose(a)/\$fell(a)**—will desugar to the same shadow-register idiom for **arch build** and to cycle-shifted term references for **arch formal**. Unbounded-liveness and general sequence operators (**##[a:b]**, **[*n]**, **throughout**, **within**, **first_match**, **s_eventually**, and strong/weak property variants) are intentionally out of scope; designs requiring them can use a dedicated SVA sidecar on the generated SystemVerilog. The rationale is the same AI-generatability contract that governs the rest of the language: keeping the verification surface small enough that an LLM can produce correct property code from a plain-English description without prior training on SVA idioms.

Two independent verification backends consume these properties. On the SystemVerilog side, Verilator [9] 5.034 **-assert** checks properties at simulation time and EBMC [33] 5.11 performs bounded model checking of the generated SystemVerilog (**ebmc -top Mod -bound N -reset "rst==1" file.sv**). On the Arch side, the **arch formal** command lowers the AST directly to SMT-LIB2 and dispatches properties to a back-end solver—z3, Boolector, or Bitwuzla—without going through Yosys or SymbiYosys. Registers become per-cycle bit-vector variables, next-state logic is encoded as an **ite** chain, and each **assert** or **cover** becomes a per-cycle disjunction checked by a single **check-sat** call. The command line is:

```
1 arch formal File.arch [--top Name]
2                               [--bound N]
3                               [--solver z3|boolector|bitwuzla]
4                               [--emit-smt out.smt2]
5                               [--timeout SEC]
```

Exit codes distinguish the three outcomes of a formal run: 0 when every property is **PROVED** (for **assert**) or **HIT** (for **cover**), 1 when any property is **REFUTED** with a counterexample or **NOT-REACHED**, and 2 when the solver returns inconclusive or times out. End-to-end verification across the two paths (SV-SVA via EBMC and direct SMT via **arch formal**) has been exercised on representative properties: a counter bound (**cnt < 201**) proved, a 4-bit overflow (**cnt != 15**) refuted with a concrete counterexample, a reachability cover (**cnt == 8**) hit at the expected witness cycle and reported **NOT-REACHED** at a lower bound, and a guard-clause contract (**valid implies written**) proved when the producer is correct and refuted when the write is missing. The three solvers agree on all ten integration tests. The scope of **arch formal v1** is flat modules with scalar signals and a single clock (no **sub-inst**, no **Vec/struct/enum**), with unsupported constructs producing clear error messages; hierarchical designs and the remaining type encodings are planned (Section 13).

8 Compile-Time Generation

Arch provides a `generate_for` / `generate_if` system for compile-time structural replication and conditional instantiation. The fused single-token keywords keep the grammar strictly LL(1) (Section 5). Unlike SystemVerilog, where the port list of a module is a fixed declaration, Arch’s `generate` operates before elaboration, so generated ports, instances, registers, connections, and assertions are indistinguishable from hand-written declarations.

Listing 14. Generate-for iteration

```

1 module SystolicArray
2   param SIZE: const = 4;
3   generate_for i in 0..SIZE
4     port data_in[i]: in SInt<8>;
5     inst pe[i]: SystolicPE
6       a <- data_in[i];
7       sum_in <- if i == 0 then 0
8         else pe[i-1].sum_out;
9     end inst pe[i]
10  end generate_for
11 end module SystolicArray

```

This capability enables expressing parameterized hardware arrays and conditional ports (via `generate_if`)—features that SystemVerilog cannot express without workarounds. The conditional-port case is particularly important: SystemVerilog’s `generate` runs *after* the module’s port list is fixed, so any configurable interface forces designers to use the preprocessor:

Listing 15. SystemVerilog forces preprocessor for conditional ports

```

1 module Cache (
2   input logic clk,
3   input logic [31:0] addr,
4   output logic [31:0] data
5   `ifdef ENABLE_DEBUG
6     , output logic [7:0] debug_state
7     , output logic      debug_valid
8   `endif
9 );

```

This is a textbook macro use that no amount of stylistic discipline can avoid in SystemVerilog—there is simply no language mechanism for it. Arch handles the same case as ordinary code with full type checking on both branches:

Listing 16. Conditional ports in Arch via `generate_if`

```

1 module Cache
2   param ENABLE_DEBUG: const = false;
3   port clk: in Clock<SysDomain>;
4   port addr: in UInt<32>;
5   port data: out UInt<32>;
6
7   generate_if ENABLE_DEBUG
8     port debug_state: out UInt<8>;
9     port debug_valid: out Bool;
10  end generate_if
11 end module Cache

```


The same mechanism extends to conditional registers, conditional sub-instances, and conditional assertions—all expressed in the source language with no separate preprocessing pass.

9 Case Study: L1 Data Cache

To demonstrate Arch’s construct composition at production scale, we implemented an 8-way set-associative write-back/write-allocate L1 data cache: $64 \text{ sets} \times 8 \text{ ways} \times 64\text{B lines} = 32 \text{ KiB}$, with a CVA6-compatible CPU interface and AXI4 memory interface. The design comprises 1,143 lines of Arch source across 12 files, generating 1,217 lines of SystemVerilog (~6% more concise). All unit and integration tests pass.

9.1 Architecture and Construct Usage

The cache exercises six distinct Arch constructs working together:

- **fsm** $\times$ **3**: A 9-state main cache controller (Idle $\rightarrow$ Lookup $\rightarrow$ Hit/Miss $\rightarrow$ Refill $\rightarrow$ Writeback), a 4-state AXI4 read burst FSM, and a 4-state AXI4 write burst FSM. The controller orchestrates the Fill and Wb FSMs via start/done pulse handshaking.
- **ram** $\times$ **3**: Tag SRAMs ($64 \times 54\text{b}$ per way, 8 instances), a data SRAM ($4096 \times 64\text{b}$ indexed by {set, way, word}), and an LRU state SRAM ($64 \times 7\text{b}$ pseudo-LRU tree). All use **simple_dual** topology with **latency** 1.
- **bus** $\times$ **2**: An AXI4 parameterized bus and a CVA6 CPU-to-cache bus, both with initiator/target perspective flipping.
- **module** $\times$ **2**: The top-level integrator and a combinational 8-way pseudo-LRU tree update module.
- **generate_for**: 8-way tag array instantiation without code duplication.
- **package**: Shared type definitions and state enums.

9.2 Design Highlights

All 8 tag ways are compared in parallel, with one-hot-to-binary encoding in 3 OR levels (10 total logic levels, optimized from 14 during development). Variable-indexed **Vec** access (**tag_rd_data[lru_victim_way]** [53:2]) eliminates manual mux trees. Tag entries are packed into 54-bit words (tag, dirty, valid) stored as **UInt<54>**.

Table 8. L1D cache source breakdown

File	Purpose	Lines
FsmCacheCtrl.arch	Main controller FSM	380
L1DCache.arch	Top-level integrator	319
FsmAxi4Wb.arch	AXI4 writeback FSM	107
FsmAxi4Fill.arch	AXI4 fill FSM	91
ModuleLruUpdate.arch	Pseudo-LRU tree	60
PkgL1d.arch	Types and enums	49
bus_axi4.arch	AXI4 bus definition	47
Ram*.arch ($\times 3$)	Tag/Data/LRU SRAMs	70
bus_dcpu.arch	CPU bus definition	20
Total		1,143

9.3 Verification

Nine C++ testbenches (1,321 lines) validate the design at both unit and integration levels. The integration testbench models AXI4 memory with a sparse `std::map` and exercises cold misses, hits, store hits/misses, and dirty evictions. The LRU testbench exhaustively covers all 128 tree states $\times$ 8 ways. Load-hit latency is 3 cycles; load-miss with clean eviction is $\sim$ 15 cycles; dirty eviction plus refill is $\sim$ 25 cycles.

9.4 Lessons

This case study illustrates several Arch principles at production scale: modular FSM composition with clear inter-FSM handshaking, first-class `ram` constructs replacing hundreds of lines of manual SRAM instantiation, `bus` abstractions making protocol interfaces reusable, and `generate_for` eliminating per-way code duplication. Three compiler bugs were found and fixed during development (Bool width inference for concat operations, `let` assign-to-existing-port syntax, and tag-hit logic level optimization), demonstrating the co-design feedback loop between language, compiler, and real hardware.

10 Case Study: AXI DMA Controller

To validate Arch across a wider range of constructs and demonstrate synthesis-quality output, we implemented a dual-channel AXI DMA controller compatible with the Xilinx PG021 specification [16]. The design supports both Simple DMA (register-triggered) and Scatter-Gather (descriptor-chained) modes. It comprises 1,042 lines of Arch source across 14 files, generating 1,176 lines of SystemVerilog ($\sim$ 11% more concise). This is the only case study exercising `bus`, `fsm`, `fifo`, `latch`, and `module` together.

10.1 Architecture

The top module integrates two symmetric DMA channels (Memory-to-Stream and Stream-to-Memory), each containing a data-path FSM, an optional Scatter-Gather descriptor engine, and a decoupling FIFO:

- **fsm $\times$ 3:** `FsmMm2s` (4 states), `FsmS2mm` (6 states), and `FsmSgEngine` (9 states implementing PG021 descriptor fetch, chain, and status writeback).
- **fifo $\times$ 2:** Synchronous depth-16 FIFOs (15 lines each) decoupling AXI from AXI-Stream timing.
- **bus $\times$ 5:** AXI4 Full, AXI4 Read-only, AXI4 Write-only, AXI4-Lite, and AXI-Stream, each expressed as a set of `handshake` channels (Section 4.5). Initiator/target perspective flipping is automatic; SV codegen flattens to individual signals.
- **latch:** Latch-based integrated clock gating cells gate each channel clock when halted, with OR logic preventing deadlock when a start signal arrives while gated.
- **module $\times$ 2:** Top-level integrator (with simple/SG combinational mux) and PG021-compatible register block (12 registers with W1C interrupt clearing).
- **package:** Shared domain definition.

A key design decision is that the data-path FSMs are shared between Simple and Scatter-Gather modes via a combinational mux—no duplicate FSMs are needed.

10.2 Synthesis Results

The generated SystemVerilog was synthesized through Yosys [17] targeting both Xilinx 7-series and SkyWater Sky130 130nm [18] to validate output quality.

Table 9. AXI DMA synthesis results

Target	Metric	Value
Xilinx 7-series	Total LUTs	913
	Flip-flops (FDRE)	993
	Estimated LCs	586
Sky130 130nm	Total area	78,134 μm^2
	Flip-flops	2,017

10.3 Timing and Power

OpenSTA [19] timing analysis with Sky130 liberty files shows a critical path of 4.478 ns, meeting timing at both 100 MHz (+5.06 ns slack) and 200 MHz (+0.06 ns slack), for a maximum achievable frequency of ~ 223 MHz on 130nm. Three rounds of critical-path optimization—guided by Yosys [17] longest-topological-path analysis—reduced logic depth from 23 levels to 18 levels (6 LUT levels after LUT6 mapping), using lookahead registers to keep final combinational paths to a single gate.

VCD-annotated power analysis from `arch sim` testbench waveforms shows 9.03 mW total at 100 MHz during active operation. Latch-based clock gating reduces idle power from 9.73 mW to ~ 0.02 mW (leakage only) when both channels are halted.

10.4 Verification

Eight C++ testbenches (2,075 lines) cover unit tests for each FSM and FIFO, register read/write and W1C interrupt clearing, clock-gating race conditions and deadlock prevention, and full integration tests including bidirectional transfers and Scatter-Gather descriptor chains with status writeback.

10.5 Thread vs. FSM Synthesis Comparison

Since the MM2S and S2MM channels are natural candidates for either a monolithic FSM or a set of concurrent `threads` (Section 4), we re-implemented both channels in thread style and synthesized the two versions side by side through Yosys `synth -flatten` to generic gates. Table 10 summarizes the result.

Table 10. Thread vs. FSM synthesis (Yosys `synth -flatten`)

Module	Style	Cells	FFs	MUXes
FsmMm2sMulti	FSM	805	109	33
ThreadMm2s (v1)	Thread	1431	97	429
ThreadMm2s (v2)	Thread	680	92	68
FsmS2mmMulti	FSM	1002	119	—
ThreadS2mm	Thread	4051	616	—

The initial thread implementation of MM2S (v1) was $1.8\times$ larger than the FSM. Root cause: four parallel threads each drove the 32-bit `ar_addr` and 32-bit `push_data` outputs, producing 4-way mux chains (429 MUX cells versus 33 in the FSM) plus a variable-by-variable multiply in the combinational path to compute burst addresses from `thread_complete[i] * burst_len_r`.

Three architectural changes, all expressible directly in Arch, produced v2:

- A single `ArIssuer` thread drives all AR outputs, eliminating the 4-way mux on the 48-bit AR channel (`ar_addr + ar_id + ar_len + ar_size + ar_burst`).
- `push_data = r_data` is hoisted to a module-level `comb` wire, since all threads push from the same source—removing the 4-way 32-bit mux on the data path.
- An incremental address register increments by `burst_len << 2` on each issue, replacing the variable-by-variable multiply with a shift-and-add.

After these changes, `ThreadMm2s v2` is **16% smaller than the FSM** (680 vs. 805 cells) with fewer flip-flops (92 vs. 109) and roughly $2\times$ as many mux cells (68 vs. 33) distributed across the 1-bit control signals (`r_ready`, `push_valid`) that the four `RCollect` threads drive in parallel—an acceptable overhead given the throughput improvement.

A separate compiler optimization—*loop-counter width inference*—contributed substantially to this result. The thread lowering pass walks the type map for each `for VAR in START..END` loop and sizes the generated state counter to the minimum width sufficient for the end expression, rather than defaulting to 32 bits. In this case, `for i in 0..burst_len_r-1` where `burst_len_r: UInt<8>` generates an 8-bit counter (`logic [7:0] _loop_cnt`) instead of 32 bits, cutting an earlier `ThreadMm2s` revision from 2660 to 1431 cells (46% reduction) before the architectural split above further reduced it to 680.

The `S2MM` thread version remains substantially larger than its FSM counterpart (4051 vs. 1002 cells) because the current design uses `fork/join` for AW+W channel parallelism inside each of four outstanding-transaction threads, plus a dedicated B-channel lock—multiplying state across threads. The same split-issuer optimization applies but has not yet been implemented.

The honest takeaway is that a thread-style design is not automatically smaller than an FSM: the naive decomposition can be $1.8\times$ to $4\times$ larger. But with the right architectural partition—one issuer thread plus N collectors, hoisted shared drivers, and incremental rather than multiplied address computation—`thread` competes with and can beat hand-written FSMs while expressing the same design in roughly one-third the lines of code.

10.6 Lessons

This case study demonstrates that Arch-generated SystemVerilog is synthesis-ready for both FPGA and ASIC targets without manual intervention. The five `bus` definitions eliminated approximately 200 lines of repetitive port declarations. The `fifo` construct reduced each FIFO to 15 lines of Arch versus ~ 80 lines of equivalent manual SystemVerilog. Clock gating via the `latch` construct proved essential for power optimization, and the deadlock-prevention pattern (OR of halt and start signals) was expressible directly in Arch’s combinational logic. The Scatter-Gather engine’s 9-state FSM—the most complex single construct in any case study—compiled and simulated correctly on the first attempt after passing `arch check`, validating the tight compile-time feedback loop.

11 Empirical Evaluation

We evaluate the Arch compiler’s correctness and expressiveness against two independent benchmark suites covering a combined 458 hardware design problems.

11.1 VerilogEval v2

VerilogEval v2 [20] is a suite of 156 Verilog design problems originally developed by NVIDIA for evaluating LLM-based hardware generation.

11.1.1 Methodology. Each problem was solved from its natural-language specification only; no reference SystemVerilog was consulted. The Arch compiler generated all SystemVerilog output, which was then compiled and simulated against the benchmark’s reference testbenches using Verilator. A problem is considered solved when Verilator reports zero mismatches between the generated design and the reference outputs.

Table 11. VerilogEval v2 benchmark results

Metric	Result
Problems solved	156 / 156 (100%)
Verilator-clean	154 / 156 (99%)
Total Arch lines	3,199
Total generated SV lines	4,518
Overall Arch/SV ratio	70.8% (~29% shorter)

11.1.2 Results. All 156 problems were solved. Of these, 154 compile and simulate cleanly under Verilator; the remaining 2 failures are defects in the benchmark dataset itself (a port name mismatch in the test harness and a mixed blocking/non-blocking assignment that Verilator rejects as illegal).

11.1.3 Code Density by Category. Table 12 breaks down line counts by design category. FSMs show the largest compression (~37% reduction) because the `fsm` construct eliminates state encoding declarations, separate `always_ff/always_comb` blocks, and `case` statement boilerplate. Combinational and simple sequential designs show roughly 1:1 ratios since there is minimal boilerplate to eliminate.

Table 12. Arch vs. generated SV line counts by category

Category	<i>n</i>	Arch	SV	Ratio
Combinational	83	~1,100	~1,300	~85%
Sequential	44	~900	~1,100	~82%
FSM	29	~1,500	~2,400	~63%
Total	156	3,199	4,518	70.8%

11.1.4 Construct Usage. Of the 156 solutions, 127 use a single `module` construct and 29 use the `fsm` construct, following a one-construct-per-file convention. File sizes range from 6–7 lines for trivial constant-output

designs to 96 lines for the most complex FSM (a bit-pattern detector with shift delay and countdown timer). Notable designs include a 16×16 toroidal Game of Life cellular automaton, a gshare branch predictor with 128-entry pattern history table, and an HDLC framing protocol FSM with 10 states.

11.1.5 Language Features Driven by Benchmark. Three problems exposed gaps that drove new language features during the benchmark:

- A `latch on ENABLE ... end latch` construct was added for problems requiring level-sensitive storage (emitting `always_latch` in SystemVerilog).
- Dual-edge flip-flop support was added via a `posedge FF + negedge FF + clock-level mux` pattern.
- Negedge-triggered latches were resolved with the new latch construct.

This feedback loop—benchmark exposes gap, language feature is added, compiler is updated, all 156 problems re-verified—illustrates the iterative co-design of the Arch language and compiler.

11.2 CVDP Benchmark

The Copilot Verilog Design Problems (CVDP) [21] benchmark provides 302 design problems of greater complexity than VerilogEval, with cocotb [15]-based testbenches and parameterized test cases that exercise designs across multiple configurations spanning five problem categories (cid002, cid003, cid004, cid007, cid016).

11.2.1 ARCH-Relevant Task Selection. The CVDP dataset mixes task types. We separate the 302 raw tasks into two groups based on the prompt’s intent:

- **ARCH-relevant (287 tasks):** the prompt asks the implementer to produce a working RTL design from a natural-language specification. This is the use case Arch is designed for. We further split this group into 221 *strict spec-to-RTL* tasks (the canonical CVDP form) and 66 *edit/complete* tasks where the prompt provides a partial design or interface and a fresh from-scratch implementation in Arch is a fair response.
- **Poor-fit (15 tasks, excluded):** the prompt’s intent is bug-fix against provided RTL, preservation of an existing implementation, or structural/optimization constraints (e.g., minimum gate count) that are not a fair test of an HDL designed for from-scratch generation. These tasks measure a different capability—refactoring or optimizing existing code—than the one Arch targets.

The 15 excluded tasks are named explicitly: in cid004, `8x3_priority_encoder_0013`, `GFCM_0003`, `bus_arbiter_0004`, `cdc_pulse_synchronizer_0013`, `cont_adder_0006`, `cont_adder_0023`, `gcd_0009`, `gcd_0015`, `gcd_0023`, `matrix_multiplier_0007`, `matrix_multiplier_0010`; in cid007, `aes_key_expansion_0001`; and in cid016, `String_to_ASCII_0001`, `apb_dsp_op_0002`, `mux_synch_0011`.

11.2.2 Methodology. Each ARCH-relevant problem was implemented in Arch from its natural-language specification, compiled to SystemVerilog, and validated against the CVDP cocotb test harnesses using Icarus Verilog as the simulation backend. Problems range from simple combinational logic to multi-state FSMs, cache controllers, linked-list data structures, signal processing pipelines, and full datapaths such as FIR filters, AXI peripherals, and sorting engines.

Table 13. CVDP benchmark results by category (ARCH-relevant subset)

Category	Spec	Edit	Total	Pass
cid002	29	65	94	94
cid003	78	0	78	78
cid004	44	0	44	44
cid007	39	0	39	39
cid016	31	1	32	32
Total	221	66	287	287 (100%)

“Spec” = strict spec-to-RTL tasks; “Edit” = edit/complete tasks where a fresh from-scratch implementation is a fair response.

11.2.3 Results. All 287 ARCH-relevant tasks pass their cocotb test harnesses, including all parameterized test cases, on the current compiler. This is a substantial improvement over earlier measurements taken during active compiler development (a snapshot in early April 2026 had 133/191 testable modules passing, 70%)—driven both by compiler fixes and by language additions described below. Reaching 287/287 required an iterative loop in which each cocotb failure was triaged into a compiler bug, a missing language feature, an Arch-source bug, or a test-runner incompatibility, and fixed accordingly.

11.2.4 Compiler Improvements Driven by CVDP. The CVDP benchmark drove several compiler fixes and language additions:

- A derived-parameter elaboration bug was fixed: expressions like `param NBW_MULT: const = DATA_WIDTH + COEFF_WIDTH` were being evaluated to literals at compile time, breaking parameterized instantiation. The fix preserves the original expression in emitted SystemVerilog when the default references other parameters.
- The `inside` set-membership operator and value-list `for` iteration (`for i in {list}`) were added to support common CVDP patterns.
- **Wrapping arithmetic operators** (`+%`, `-%`, `*%`) were added for modular arithmetic at the operand width without IEEE 1800 §11.6 widening. Result width is `max(W(lhs), W(rhs))`; SV emission uses `W'(lhs op rhs)` size casts. This eliminated the `.trunc<N>()/.sext<N>()/.zext<N>()` boilerplate that caused simulator width-overflow failures in MAC and accumulator designs, converting six previously failing tests to passing across the `sgd_linear_regression`, `load_store_unit`, `low_pass_filter`, `digital_dice_roller`, `dig_stopwatch`, and `apb_dsp_op` modules.
- Reset syntax was standardized to the `=>` form (`reset rst => 0`), and a latch codegen bug was fixed (blocking = instead of `<=` in `always_latch`).

11.2.5 Combined Benchmark Summary. Across both benchmarks, the Arch compiler has been validated against 458 hardware design problems spanning combinational logic, sequential circuits, FSMs, pipelines, caches, signal processing blocks, AXI peripherals, and sorting engines. Table 14 summarizes the combined results.

Table 14. Combined benchmark results

Benchmark	Problems	In-scope	Pass Rate [‡]
VerilogEval v2	156	156	154 (99%) [†]
CVDP	302	287	287 (100%) [§]
Combined	458	443	441 (99.5%)

[†]The 2 failures are due to port-name bugs in the reference test benches, not defects in the Arch-generated designs; the effective pass rate is 156/156 (100%).

[§]15 of the 302 CVDP tasks are excluded as poor-fit (bug-fix, preserve-existing-RTL, or optimization tasks; Section 11.2). 287 ARCH-relevant tasks remain, all pass.

[‡]These pass rates were measured while the compiler was actively under development and bugs were being fixed concurrently; the numbers are indicative rather than definitive. Re-verification on a frozen compiler release is planned.

12 Related Work

?? compares Arch with the major existing HDLs across the dimensions Arch was designed to address. The remainder of this section discusses each language family in turn.

*Scala traits, not hardware-level contracts with hook bindings. [†]Expressed implicitly via guarded atomic rules; no explicit pipeline/FSM construct. [‡] $\gg N$ inline retiming and ΘN numbered stages, but no hazard management. [§]Verilator—open-source cycle-accurate compiled simulator for SystemVerilog. ^{||}`arch sim` independently generates compiled C++ models; native LLVM IR compilation is planned.

Traditional HDLs. Verilog/SystemVerilog [1] and VHDL [2] have served the industry for decades but predate modern type systems and AI-assisted workflows. SystemVerilog remains the industry standard but provides no compile-time guarantees against latches, multiply-driven nets, or CDC violations; its `generate` cannot add ports; X/Z semantics require 4-state simulation; and the irregular syntax (different block closers: `end`, `endmodule`, `endcase`, `endfunction`) creates structural ambiguity for both humans and LLMs. SystemVerilog 2017 added assertions (SVA) and constrained randomization but did not address the core RTL safety issues Arch targets.

Enhanced RTL Semantics. RTL++ [22] proposed elevating the RTL abstraction level by introducing pipelined register variables with formally defined execution semantics (structural operational semantics) and a Register-transfer Finite State Machine (RFSM) model for synthesis. Arch builds on these ideas—the `pipe_reg` construct directly realizes the pipelined register variable concept, and Arch’s first-class `pipeline` and `fsm` constructs extend the RFSM model with compiler-generated hazard logic. Where RTL++ defined enhanced semantics within the existing C++/SystemC framework, Arch implements them as native language constructs with static type checking and deterministic code generation, targeting the AI-assisted design workflow that was not yet practical in 2005.

Embedded DSLs. Chisel [3], SpinalHDL [4], Amaranth [5], PyRTL [6], and Clash [23] embed RTL in host languages. Chisel provides rich parameterization via Scala and targets the FIRRTL intermediate representation, but inherits Scala’s JVM runtime, implicit conversions, and build system complexity; its pipelines, FIFOs, and arbiters are library-defined rather than compiler-verified, and Scala traits offer type-level abstraction without Arch’s hardware-specific hook mechanism for injecting custom logic into compiler-generated constructs. SpinalHDL provides strong typing, CDC checking, and a well-developed

FSM library within a Scala DSL, but pipelines, FIFOs, and arbiters remain library patterns and the Scala host language creates the same onboarding barrier as Chisel. Amaranth (formerly nMigen) uses Python and provides single-driver enforcement and an absence of implicit latches, but its Python embedding means dynamic typing at the meta-level and no compile-time clock domain tracking. All embedded DSLs inherit host-language complexity and do not elevate micro-architectural constructs to first-class status.

Standalone Modern HDLs. Spade [24] is a recent standalone HDL from Linköping University and Munich University of Applied Sciences that shares Arch’s rejection of host-language embedding. Spade takes inspiration from Rust: it is expression-based, has global type inference, rich product and sum types, and linear typechecking for resources such as memory ports. Its headline construct is a first-class pipeline in which `reg` tokens separate stages and the compiler inserts the required synchronizing flip-flops, making re-timing and re-pipelining trivial. This is the closest cousin to Arch’s `pipe_reg` mechanism among modern HDLs. Where Arch and Spade diverge is in scope and design philosophy. Spade’s first-class constructs are pipelines, memories, and registers; FSMs, FIFOs, arbiters, register files, and buses are expressed as ordinary user code built on those primitives, and clock domains are not part of the type system, so CDC correctness still depends on external lint tools. Arch in contrast provides a larger catalog of first-class constructs (`fsm`, `fifo`, `arbiter`, `regfile`, `synchronizer`, `bus`, `thread`, and others) and lifts clock and reset to parameterized types so CDC and RDC are typing rules rather than external analyses. The two languages also make opposite AI-readability choices: Spade favors concise Rust-like syntax with inferred types, while Arch’s contract requires explicit widths, domains, directions, and an LL(1) grammar so that an LLM can produce a complete type-correct program from a natural-language description without needing to simulate an inference engine.

Rule-Based and Pipeline-Centric HDLs. Bluespec SystemVerilog (BSV) [25] uses guarded atomic actions (rules) as its fundamental concurrency primitive: each rule fires atomically when its guard is true, and the compiler schedules rules to avoid conflicts. This model excels at expressing complex microarchitectures with many interacting concurrent behaviors—out-of-order processors, for instance, can be more concise once the rule model is internalized—and Bluespec’s type system, including typeclasses and polymorphism, is richer than Arch’s. However, the scheduling model introduces a fundamental predictability problem: whether two rules fire together in a given cycle is a compiler decision that can be surprising and difficult to debug. Designers influence scheduling via `descending_urgency` pragmas rather than controlling it directly, the generated Verilog is often verbose with mangled internal names, and the steep learning curve limits adoption. Transaction-Level Verilog (TL-Verilog) [26] extends SystemVerilog with a pipeline-centric model where signals carry implicit stage context via `»N` notation; retiming is concise (`»2$operand` replaces Arch’s explicit `pipe_reg operand_d2: operand stages 2`) and restructuring pipeline depth requires minimal edits. However, TL-Verilog leaves hazard management entirely to the designer (stall propagation, flush logic, and data forwarding are manual) and does not address FSMs, bus protocols, clock-domain crossings, arbitration, or resource locking. Its toolchain depends on the proprietary Sandpiper compiler. Arch differs from both by providing explicit, predictable constructs at each abstraction level—from raw RTL (`comb/seq`) through micro-architectural primitives (`pipeline/fsm/fifo`) to protocol-level abstractions (`bus/thread`)—with deterministic, readable SystemVerilog output throughout.

High-Level Synthesis. HLS tools (Vitis HLS, Catapult, Bambu [27]) synthesize RTL from C/C++ or SystemC [28] descriptions. SystemC provides cycle-accurate modeling and transaction-level abstractions within a C++ framework, but inherits the complexity of a general-purpose host language and relies on an event-driven simulation kernel. HLS operates at a higher abstraction level than Arch; the designer surrenders control over micro-architectural structure in exchange for productivity. Arch targets designers who require cycle-accurate control over their hardware while benefiting from first-class constructs that eliminate the boilerplate HLS was designed to avoid.

CIRCT/MLIR. The CIRCT project [29] provides MLIR-based infrastructure for hardware compilation, including dialects for FIRRTL, FSM, and handshake circuits. Arch plans integration with CIRCT as a backend target (AIR $\rightarrow$ CIRCT dialects $\rightarrow$ SV/netlist) while maintaining its own frontend type system and compiler for the primary compilation path.

AI for Hardware. Recent work on LLM-based hardware generation [30–32] has demonstrated promise but is hampered by the irregular syntax and implicit failure modes of existing HDLs. Arch is the first HDL designed with AI generatability as a primary constraint.

13 Future Work

Several language features are specified but not yet implemented in the compiler.

Transaction-Level Modeling. Arch specifies Transaction Level Modeling as a first-class abstraction above RTL, where modules communicate by calling methods on typed `bus` interfaces rather than driving individual signals cycle-by-cycle. The `blocking_tlm_method` construct (v1, shipped) collapses a request/response handshake pair into a single method declaration: an initiator writes `let val = m.read(addr);` inside a `thread` body, and the compiler generates the wire protocol, call-site state machine, and target-side FSM automatically. The `credit_channel` sub-construct (shipped) handles stateful credit-based flow control. Future TLM work includes `out_of_order` response routing (auto-route responses by ID tag) and RTL-backed cycle-accurate method implementations; the multi-outstanding concurrency pattern is already expressible today via `generate for` threads + `lock` resource arbitration + ID-tagged responses, without dedicated TLM concurrency modes.

Additional Planned Features. Other specified but not yet implemented constructs include: `crossbar` (switching fabric), `scoreboard` and `reorder_buf` (out-of-order execution support), and `pqueue` (hardware priority queue). A richer `cam` surface (`kind: ternary|associative`, LRU/FIFO replacement policies, value payload, and pipelined lookup) is specified as aspirational; the v1 combinational binary-match `cam` is shipped. A `multicycle` register annotation (`reg r: UInt<32> multicycle 3 reset rst=0;`) is planned for designs where the combinational path feeding a register has a multi-cycle timing budget: the register remains a single flop but the compiler emits SDC `set_multicycle_path` constraints, optional formal assertions verifying the timing assumption, and runtime valid-tracking for `-check-uninit` to flag reads before settling. Tristate and bidirectional I/O (`Tristate<T>` type plus `tristate` block) are planned for pad-level I/O such as I2C and GPIO, with SV emitting `inout` plus Z-drivers and simulation decomposing to `_out/_oe/_in`. RDC checking (cross-clock-domain async reset, cross-async-reset-domain data paths, reset-driven clock gating, reconvergent synchronizers, and combiner-derived reset glitches) is already implemented for `module`

Manuscript submitted to ACM

constructs. Remaining RDC work includes extending data-flow tracing across instance boundaries and adding synchronizer/FIFO-internal path checking. CDC analysis will continue to be strengthened toward feature parity with commercial CDC tools. Package-scoped modules are also planned: `inst a: PkgName::Module` will allow modules to live inside packages with compile-time name resolution, eliminating SystemVerilog’s flat global module namespace and the naming collisions it produces in large designs. The formal backend (Section 7.4) now supports hierarchical designs with one level of sub-instance nesting (bottom-up flattening of `let/comb/reg/seq` sub-modules). The property language has gained lightweight temporal sugar—`a | => b, ##N expr, past(expr, N), $rose(a)/$fell(a)`—all shipped in phases 1 and 2. Remaining formal work includes deeper hierarchical support, `Vec/struct/enum` encodings, multi-clock specifications, `assume` statements, and k -induction for unbounded proofs. Optional UPF/CPF power-intent integration for low-power SoCs is a longer-term consideration. CIRCT/MLIR backend integration is planned as a secondary export path (`AIR` $\rightarrow$ `CIRCT dialects` $\rightarrow$ `SV/netlist`). A fully native LLVM IR simulation path, bypassing Verilator for higher throughput with structurally guaranteed deterministic parallel execution, is a longer-term goal enabled by the language’s structural invariants.

14 Conclusion

We have presented Arch, a hardware description language that addresses three fundamental gaps in the current HDL landscape: the absence of first-class micro-architectural constructs in mainstream HDLs, the inability of existing type systems to catch entire categories of hardware bugs at compile time, and the lack of any HDL designed for AI-assisted code generation.

Arch’s type system tracks four independent safety dimensions (bit widths, clock domains, port directions, signal ownership) and converts latches, multiply-driven nets, CDC violations, and width mismatches into compile-time errors. Its integrated `arch sim` command provides cycle-accurate simulation via independently generated compiled C++ models, with waveform output, assertion evaluation, and CDC randomization. Its AI-generatability contract—uniform schema, LL(1) grammar, named endings, directional arrows, and `todo!`—enables LLMs to produce correct hardware code without fine-tuning.

Arch is under active development with plans for open-source release under the Apache 2.0 license.

References

- [1] IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language. *IEEE Std 1800-2017*, 2018.
- [2] IEEE Standard VHDL Language Reference Manual. *IEEE Std 1076-2019*, 2019.
- [3] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzynek, and K. Asanović. Chisel: Constructing hardware in a Scala embedded language. In *Proc. DAC*, pages 1216–1225, 2012.
- [4] C. Papon. SpinalHDL: An alternative hardware description language. <https://spinalhdl.github.io/SpinalDoc-RTD/>, 2016.
- [5] Amaranth HDL Project. <https://amaranth-lang.org/>, 2023.
- [6] T. Clow, J. Valamehr, J. Clemons, and T. Sherwood. PyRTL: A Python framework for register-transfer level hardware design. In *Proc. FCCM*, 2017.
- [7] M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
- [8] GitHub Copilot. <https://github.com/features/copilot>, 2023.
- [9] W. Snyder. Verilator: open-source SystemVerilog simulator and lint system. <https://www.veripool.org/verilator/>, 2024.
- [10] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley Professional, 2003.
- [11] N. S. Mathews and M. Nagappan. Test-driven development for code generation. In *Proc. 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024. arXiv:2402.13521.

- [12] S. Williams. Icarus Verilog. <https://steveicarus.github.io/iverilog/>, 2024.
- [13] J. Bennett. Building a loosely timed SoC model with OSCI TLM 2.0. Embecosm Application Note 6, Issue 1, 2013. <https://www.embecosm.com/appnotes/ean6/>.
- [14] S. Zhao, S. Yan, and Y. Feng. Practical approach using a formal app to detect X-optimism related RTL bugs. In *Proc. Design and Verification Conference (DVCon)*, 2014.
- [15] B. Rosser et al. cocotb: A coroutine-based cosimulation testbench environment for verifying VHDL and SystemVerilog RTL in Python. <https://www.cocotb.org/>, 2024.
- [16] Xilinx (AMD). *AXI DMA v7.1: LogiCORE IP Product Guide*. PG021, 2022.
- [17] C. Wolf. Yosys Open SYnthesis Suite. <https://yosyshq.net/yosys/>, 2024.
- [18] T. Edwards et al. SkyWater SKY130 open-source PDK. Google/SkyWater Technology, <https://skywater-pdk.readthedocs.io/>, 2023.
- [19] J. Cherry and T. Ajayi. OpenSTA: open-source static timing analyzer. The OpenROAD Project, <https://github.com/The-OpenROAD-Project/OpenSTA>, 2024.
- [20] M. Liu, N. Pinckney, B. Khailany, and H. Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *Proc. ICCAD*, 2023.
- [21] Y. Tsai, M. Liu, and H. Ren. CVDp: Copilot Verilog Design Problems benchmark suite. <https://github.com/NVlabs/cvdp>, 2024.
- [22] S. Zhao and D. D. Gajski. Defining an enhanced RTL semantics. In *Proc. Design, Automation and Test in Europe (DATE)*, pages 548–553, 2005.
- [23] C. Baaij, M. Kooijman, J. Kuper, A. Boeijink, and M. Gerards. C_λASH: Structural descriptions of synchronous hardware using Haskell. In *Proc. Euromicro DSD*, pages 714–721, 2010.
- [24] F. Skarman and O. Gustafsson. Spade: An HDL inspired by modern software languages. In *Proc. 32nd Int. Conf. on Field-Programmable Logic and Applications (FPL)*, 2022.
- [25] R. Nikhil. Bluespec SystemVerilog: efficient, correct RTL from high-level specifications. In *Proc. MEMOCODE*, pages 69–70, 2004.
- [26] S. Hoover. Timing-abstract circuit design in transaction-level Verilog. In *Proc. ICCD*, pages 576–579, 2017.
- [27] C. Pilato and F. Ferrandi. Bambu: A modular framework for the high level synthesis of memory-intensive applications. In *Proc. FPL*, pages 1–4, 2013.
- [28] IEEE Standard for Standard SystemC Language Reference Manual. *IEEE Std 1666-2023*, 2023.
- [29] CIRCT: Circuit IR Compilers and Tools. <https://circt.llvm.org/>, 2024.
- [30] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg. VeriGen: A large language model for Verilog code generation. *arXiv:2308.00708*, 2023.
- [31] M. Liu, T. Ene, R. Kirber, et al. ChipNeMo: Domain-adapted LLMs for chip design. *arXiv:2311.00176*, 2023.
- [32] J. Blocklove, S. Garg, R. Karri, and H. Pearce. Chip-Chat: Challenges and opportunities in conversational hardware design. *arXiv:2305.13243*, 2023.
- [33] D. Kroening, P. Schrammel, and M. Tautschnig. EBMC: The Enhanced Bounded Model Checker. <https://www.cprover.org/ebmc/>, 2024.